University of East London School of Architecture, Computing and Engineering (ACE)
Author: Dr Fahimeh Jafari (f.jafari@uel.ac.uk) 25-26

# Tutorial 7: KDDCup'99 Analysis using Spark SQL

CN7031 - Big Data Analytics

Dr Fahimeh Jafari (f.jafari@uel.ac.uk)

LEARNING OUTCOMES: After completing this tutorial, you should:

- Have gotten hands-on experience in using Spark SQL for data analysis.
- Understand the real case studies applied in the Big Data platform for analysis.
- Visualize the outcomes of queries in the graphical representations.

## Tutorial Submission [OPTIONAL]

You can submit your `.html` file through the submission link provided in Moodle. Task 5 explains how to extract your `html` file through both Google Colab and Jupyter in VMWare.

## Task 1: Understanding KDDCup Dataset

KDDCup-99 is the data set used for The Third International Knowledge Discovery and Data Mining Tools Competition, which was held in conjunction with KDD-99, the Fifth International Conference on Knowledge Discovery and Data Mining. The competition task was to build a network intrusion detector, a predictive model capable of distinguishing between 'bad' connections, called intrusions or attacks, and 'good' normal connections. This database contains a standard set of data to be audited, which includes a wide variety of intrusions simulated in a military network environment. This dataset has 22 types of bad connections, namely, `back, buffer_overflow, ftp_write, guess_passwd, imap, ipsweep, land, loadmodule, multihop, neptune, nmap, perl, phf, pod, portsweep, rootkit, satan, smurf, spy, teardrop, warezclient, warezmaster.`

a) As big data specialists, firstly, we would like to read and understand its features, and then apply modeling techniques.
b) The features are described here.
   http://kdd.ics.uci.edu/databases/kddcup99/task.html
c) You can find more descriptions about the dataset here.
   http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html

**Step 1:** Understand the features of the dataset.

There are 42 features as below:

```
1- duration: continuous.
2- protocol_type: symbolic.
3- service: symbolic.
```

```
4- flag: symbolic.
5- src_bytes: continuous.
6- dst_bytes: continuous.
7- land: symbolic.
8- wrong_fragment: continuous.
9- urgent: continuous.
10-  host: continuous.
11-  num_failed_logins: continuous.
12-  logged_in: symbolic.
13-  num_compromised: continuous.
14-  root_shell: continuous.
15-  su_attempted: continuous.
16-  num_root: continuous.
17-  num_file_creations: continuous.
18-  num_shells: continuous.
19-  num_access_files: continuous.
20-  num_outbound_cmds: continuous.
21-  is_host_login: symbolic.
22-  is_guest_login: symbolic.
23-  count: continuous.
24-  srv_count: continuous.
25-  serror_rate: continuous.
26-  srv_serror_rate: continuous.
27-  rerror_rate: continuous.
28-  srv_rerror_rate: continuous.
29-  same_srv_rate: continuous.
30-  diff_srv_rate: continuous.
31-  srv_diff_host_rate: continuous.
32-  dst_host_count: continuous.
33-  dst_host_srv_count: continuous.
34-  dst_host_same_srv_rate: continuous.
35-  dst_host_diff_srv_rate: continuous.
36-  dst_host_same_src_port_rate: continuous.
37-  dst_host_srv_diff_host_rate: continuous.
38-  dst_host_serror_rate: continuous.
39-  dst_host_srv_serror_rate: continuous.
40-  dst_host_rerror_rate: continuous.
41-  dst_host_srv_rerror_rate: continuous.
42-  connection_status:
   back,buffer_overflow,ftp_write,guess_passwd,imap,ipsweep,land,loadm
   odule,multihop,neptune,nmap,normal,perl,phf,pod,portsweep,rootkit,s
   atan,smurf,spy,teardrop,warezclient,warezmaster.
```

- The whole overview of the features are as follows:

| *feature name* | *description* | *type* |
|---|---|---|
| duration | length (number of seconds) of the connection | continuous |
| protocol_type | type of the protocol, e.g. tcp, udp, etc. | discrete |
| service | network service on the destination, e.g., http, telnet, etc. | discrete |
| src_bytes | number of data bytes from source to destination | continuous |
| dst_bytes | number of data bytes from destination to source | continuous |
| flag | normal or error status of the connection | discrete |
| land | 1 if connection is from/to the same host/port; 0 otherwise | discrete |
| wrong_fragment | number of “wrong” fragments | continuous |
| urgent | number of urgent packets | continuous |
| host | number of “host” indicators | continuous |

| | | |
|---|---|---|
| num_failed_logins | number of failed login attempts | continuous |
| logged_in | 1 if successfully logged in; 0 otherwise | discrete |
| num_compromised | number of "compromised" conditions | continuous |
| root_shell | 1 if root shell is obtained; 0 otherwise | discrete |
| su_attempted | 1 if "su root" command attempted; 0 otherwise | discrete |
| num_root | number of "root" accesses | continuous |
| num_file_creations | number of file creation operations | continuous |
| num_shells | number of shell prompts | continuous |
| num_access_files | number of operations on access control files | continuous |
| num_outbound_cmds | number of outbound commands in an ftp session | continuous |
| is_host_login | 1 if the login belongs to the "host" list; 0 otherwise | discrete |
| is_guest_login | 1 if the login is a guest login; 0 otherwise | discrete |
| count | number of connections to the same host as the current connection in the past two seconds | continuous |
| | *Note: The following features refer to these same-host connections.* | |
| serror_rate | % of connections that have "SYN" errors | continuous |
| rerror_rate | % of connections that have "REJ" errors | continuous |
| same_srv_rate | % of connections to the same service | continuous |
| diff_srv_rate | % of connections to different services | continuous |
| srv_count | number of connections to the same service as the current connection in the past two seconds | continuous |
| | *Note: The following features refer to these same-service connections.* | |
| srv_serror_rate | % of connections that have "SYN" errors | continuous |
| srv_rerror_rate | % of connections that have "REJ" errors | continuous |
| srv_diff_host_rate | % of connections to different hosts | continuous |
| connection_status | The status of the connection (class label) | Symbol |

- A sample of data is:

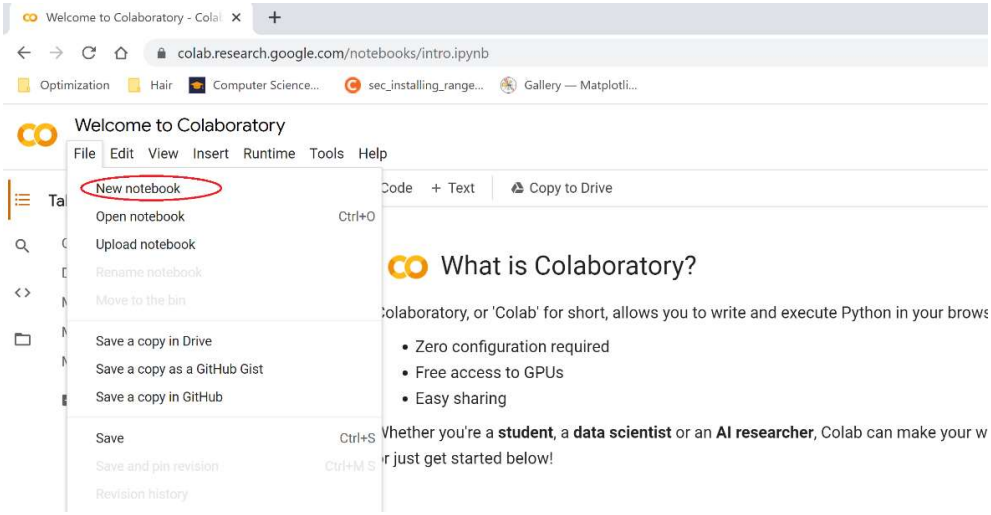
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 3 | 0 | 0 | 1 | 0.33 | 1 | 0 | 0.67 | 3 | 255 | 1 | 0 | 0.33 | 0.15 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 13 | 255 | 1 | 0 | 0.08 | 0.15 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 23 | 255 | 1 | 0 | 0.04 | 0.14 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 33 | 255 | 1 | 0 | 0.03 | 0.13 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 43 | 255 | 1 | 0 | 0.02 | 0.12 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 53 | 255 | 1 | 0 | 0.02 | 0.11 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 63 | 255 | 1 | 0 | 0.02 | 0.09 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 73 | 255 | 1 | 0 | 0.01 | 0.07 | 0 | 0 | 1 | 0.79 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 83 | 255 | 1 | 0 | 0.01 | 0.07 | 0 | 0 | 1 | 0.81 | normal. |
| tcp | http | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 255 | 1 | 0 | 1 | 0.07 | 0 | 0 | 1 | 0.85 | normal. |
| icmp | eco_i | SF | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 29 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 2 | 2 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | ipsweep. |
| icmp | eco_i | SF | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 5 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 2 | 12 | 1 | 0 | 1 | 0.5 | 0 | 0 | 0 | 0 | ipsweep. |
| icmp | eco_i | SF | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 10 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 2 | 22 | 1 | 0 | 1 | 0.5 | 0 | 0 | 0 | 0 | ipsweep. |
| icmp | eco_i | SF | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 14 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 2 | 32 | 1 | 0 | 1 | 0.5 | 0 | 0 | 0 | 0 | ipsweep. |
| icmp | eco_i | SF | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 19 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 2 | 42 | 1 | 0 | 1 | 0.5 | 0 | 0 | 0 | 0 | ipsweep. |
| icmp | eco_i | SF | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 23 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 2 | 52 | 1 | 0 | 1 | 0.5 | 0 | 0 | 0 | 0 | ipsweep. |

## Task 2: Getting Ready

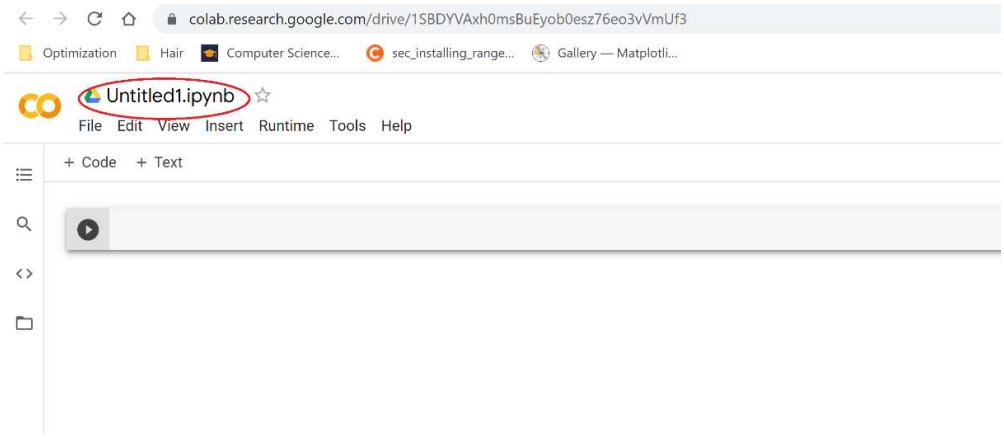
As explained in the previous tutorial, you can execute PySpark commands using Jupyter in VMWare OR Google *Colab* which is an online platform. Please go to the correct section due to your notebook.

### Google Colab

- Open the Google Colab through https://colab.research.google.com/
- Open a new notebook to type the commands.




- Click on the name box (see the red circle) and rename your file to `KDDCup`.



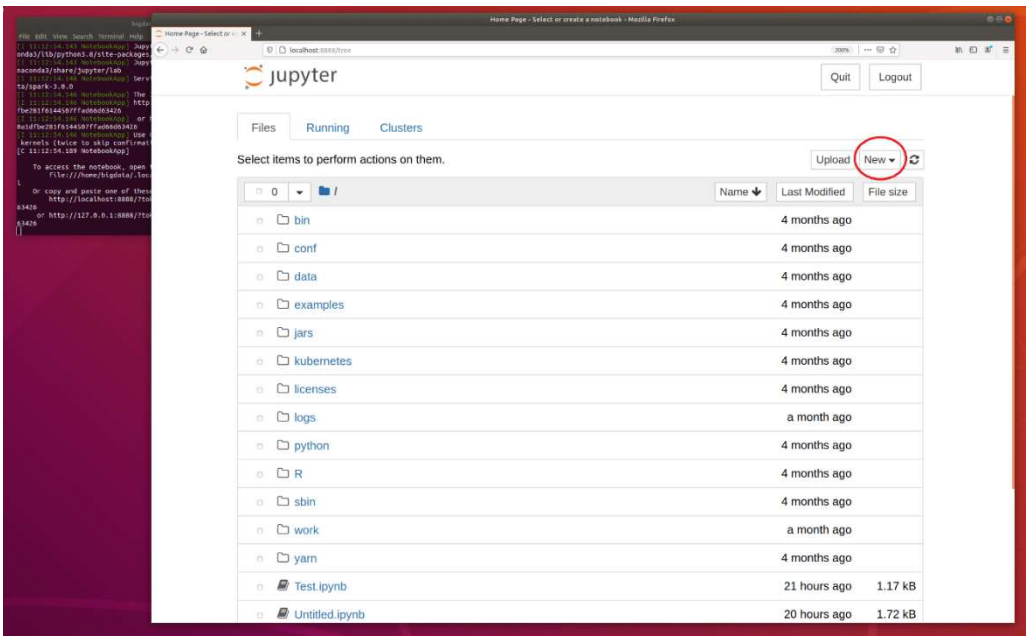

- Type and run the following commands in the editor to install and lunch Spark.

```
# [1] download and install pyspark in Google Colab
!pip3 install pyspark
```

**Now, go to Task 3**

### Jupyter in VMWare

- Run VMWare machine with `Big Data-Ubuntu`
- Open the terminal and go to Spark directory by typing `cd $SPARK_HOME`
- Type `pyspark` in the terminal to launch PySpark and Jupyter.
- Open a new notebook in Jupyter through `New->Python3` (red cycle in the screenshot). Now, your editor is ready for programming.




## Task 3: Reading Data

**Step 1**:

Go to the editor and type the following commands to retrieve KDDCup data using URL. The `urllib.request` module defines functions and classes which help in opening URLs.

```
import urllib.request
urllib.request.urlretrieve("http://kdd.ics.uci.edu/databases/kddcup99/kddcup.data.gz", "kddcup.data.gz")
```

**Note: If the above URL doesn't work, you can try the following one:**

```
urllib.request.urlretrieve("https://www.dropbox.com/s/qz62t2oyllkl32s/kddcup.data_10_percent.gz?dl=1", "kddcup.data_10_percent.gz")
```

The entry point into all functionality in Spark is the SparkSession class. So, type the following commands to create a basic SparkSession.

```
from pyspark.sql import SparkSession
spark = SparkSession \
    .builder \
    .appName("Python Spark SQL basic example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
```

Create DataFrame based on the content of KDDCup file.

```
df = spark.read.csv("./kddcup.data.gz")
```

**Note: if you have used the second URL, you should create DataFrame using the following command.**

```
df = spark.read.csv("./kddcup.data_10_percent.gz”)
```

```
df.show()
```

```
+---+---+----+---+---+----+---+---+---+---+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
|_c0|_c1| _c2|_c3|_c4| _c5|_c6|_c7|_c8|_c9|_c10|_c11|_c12|_c13|_c14|_c15|_c16|_c17|_c18|_c19|_c20|_c21|_c22|_c23|_c24|_c25|_c26|_c27|_c28|_c29|
+---+---+----+---+---+----+---+---+---+---+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
|  0|tcp|http| SF|181|5450|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   8|   8|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|239| 486|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   8|   8|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|235|1337|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   8|   8|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|219|1337|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   6|   6|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|217|2032|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   6|   6|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|217|2032|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   6|   6|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|212|1940|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   1|   2|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|159|4087|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   5|   5|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|210| 151|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   8|   8|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|212| 786|  0|  0|  0|  1|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   8|   8|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|210| 624|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|  18|  18|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|177|1985|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   1|   1|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|222| 773|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|  11|  11|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|256|1169|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   4|   4|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|241| 259|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   1|   1|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|260|1837|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|  11|  11|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|241| 261|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   2|   2|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|257| 818|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|  12|  12|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|233| 255|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   2|   8|0.00|0.00|0.00|0.00|1.00|0.00|
|  0|tcp|http| SF|233| 504|  0|  0|  0|  0|   0|   1|   0|   0|   0|   0|   0|   0|   0|   0|   0|   0|   7|   7|0.00|0.00|0.00|0.00|1.00|0.00|
+---+---+----+---+---+----+---+---+---+---+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
only showing top 20 rows
```

```
df.columns
```

```
['_c0',
 '_c1',
 '_c2',
 '_c3',
 '_c4',
 '_c5',
 '_c6',
 '_c7',
 '_c8',
 '_c9',
 '_c10',
 '_c11',
 '_c12',
 '_c13',
 '_c14',
 '_c15',
 '_c16',
 '_c17',
 '_c18',
 '_c19',
 '_c20',
 '_c21',
 '_c22',
 '_c23',
```

Rename the columns as follows:

```
df2 = df.withColumnRenamed("_c0","duration") \
    .withColumnRenamed("_c1","protocol_type")\
    .withColumnRenamed("_c2","service")\
    .withColumnRenamed("_c3","flag")\
    .withColumnRenamed("_c4","src_bytes")\
    .withColumnRenamed("_c5","dst_bytes")\
    .withColumnRenamed("_c6","land")\
    .withColumnRenamed("_c7","wrong_fragment")\
    .withColumnRenamed("_c8","urgent")\
    .withColumnRenamed("_c9","host")\
    .withColumnRenamed("_c10","num_failed_logins")\
    .withColumnRenamed("_c11","logged_in")\
    .withColumnRenamed("_c12","num_compromised")\
    .withColumnRenamed("_c13","root_shell")\
    .withColumnRenamed("_c14","su_attempted")\
    .withColumnRenamed("_c15","num_root")\
    .withColumnRenamed("_c16","num_file_creations")\
    .withColumnRenamed("_c17","num_shells")\
    .withColumnRenamed("_c18","num_access_files")\
    .withColumnRenamed("_c19","num_outbound_cmds")\
    .withColumnRenamed("_c20","is_host_login")\
    .withColumnRenamed("_c21","is_guest_login")\
    .withColumnRenamed("_c22","count")\
    .withColumnRenamed("_c23","srv_count")\
    .withColumnRenamed("_c24","serror_rate")\
    .withColumnRenamed("_c25","srv_serror_rate")\
    .withColumnRenamed("_c26","rerror_rate")\
    .withColumnRenamed("_c27","srv_rerror_rate")\
    .withColumnRenamed("_c28","same_srv_rate")\
```

```
    .withColumnRenamed("_c29","diff_srv_rate")\
    .withColumnRenamed("_c30","srv_diff_host_rate")\
    .withColumnRenamed("_c31","dst_host_count")\
    .withColumnRenamed("_c32","dst_host_srv_count")\
    .withColumnRenamed("_c33","dst_host_same_srv_rate")\
    .withColumnRenamed("_c34","dst_host_diff_srv_rate")\
    .withColumnRenamed("_c35","dst_host_same_src_port_rate")\
    .withColumnRenamed("_c36","dst_host_srv_diff_host_rate")\
    .withColumnRenamed("_c37","dst_host_serror_rate")\
    .withColumnRenamed("_c38","dst_host_srv_serror_rate")\
    .withColumnRenamed("_c39","dst_host_rerror_rate")\
    .withColumnRenamed("_c40","dst_host_srv_rerror_rate")\
    .withColumnRenamed("_c41","connection_status")
```

```
df2.columns
```

```
['duration',
 'protocol_type',
 'service',
 'flag',
 'src_bytes',
 'dst_bytes',
 'land',
 'wrong_fragment',
 'urgent',
 'host',
 'num_failed_logins',
 'logged_in',
 'num_compromised',
 'root_shell',
 'su_attempted',
 'num_root',
 'num_file_creations',
 'num_shells',
 'num_access_files',
 'num_outbound_cmds',
 'is_host_login',
 'is_guest_login',
 'count',
 'srv_count',
 'serror_rate',
 'srv_serror_rate',
 'rerror_rate',
 'srv_rerror_rate',
 'same_srv_rate',
 'diff_srv_rate',
```

## Task 4: SQL Queries and Visualisation

To create SQL queries, you need to first register the DataFrame as a SQL temporary view and then define the queries on the view.

```
df2.createOrReplaceTempView("KddView")
sqlDF = spark.sql("SELECT * FROM KddView")
sqlDF.show()
```

Now, you can make some analytical queries on KDDCup99 dataset using Spark SQL.

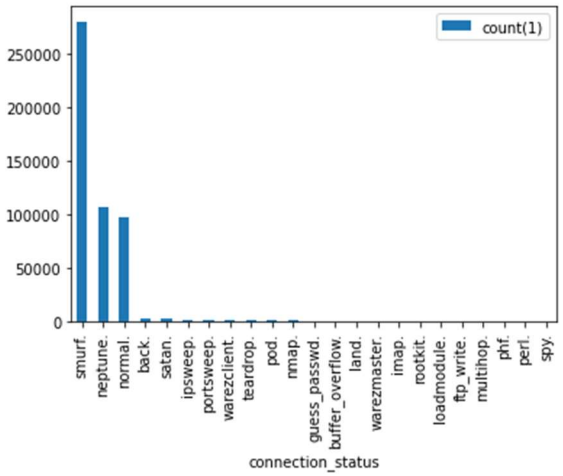
**Query 1:** List the number of connections for different connections' statuses

```
sqlDF = spark.sql("SELECT connection_status, count(*) FROM KddView
GROUP BY connection_status")
sqlDF.show()
```

```
+-----------------+--------+
|connection_status|count(1)|
+-----------------+--------+
|     warezmaster.|      20|
|           smurf.|  280790|
|             pod.|     264|
|            imap.|      12|
|            nmap.|     231|
|    guess_passwd.|      53|
|         ipsweep.|    1247|
|       portsweep.|    1040|
|           satan.|    1589|
|            land.|      21|
|      loadmodule.|       9|
|       ftp_write.|       8|
| buffer_overflow.|      30|
|         rootkit.|      10|
|     warezclient.|    1020|
|        teardrop.|     979|
|            perl.|       3|
|             phf.|       4|
|        multihop.|       7|
|         neptune.|  107201|
+-----------------+--------+
only showing top 20 rows
```

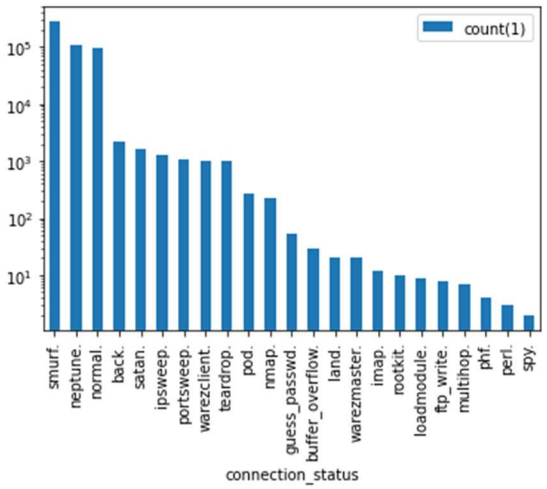
You can present the results visually by following code:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
pandas_df = sqlDF.toPandas()
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='connection_status', y='count(1)', kind = 'bar')
```




There are two main reasons to use logarithmic scales in charts and graphs. The first is to respond to skewness towards large values; i.e., cases in which one or a few points are much larger than the bulk of the data. The second is to show percent change or multiplicative factors. Due to the first reason, we pass **logy** to get a log-scale Y axis as follows:

```
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='connection_status', y='count(1)', kind = 'bar', logy=True)
```



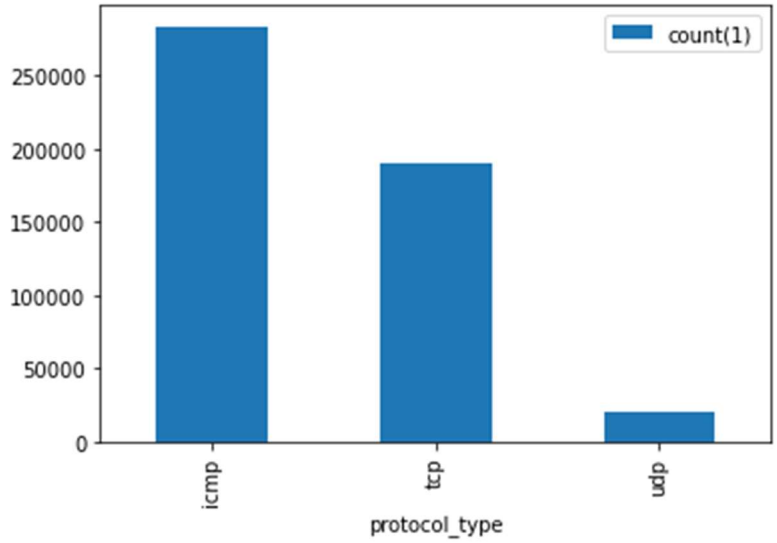

## Query 3: List the number of connections.

```
sqlDF = spark.sql("SELECT protocol_type, count(*) FROM KddView GROUP
BY protocol_type")
sqlDF.show()
```

```
+-------------+--------+
|protocol_type|count(1)|
+-------------+--------+
|          tcp|  190065|
|          udp|   20354|
|         icmp|  283602|
+-------------+--------+
```

Extract the visual presentation as below.

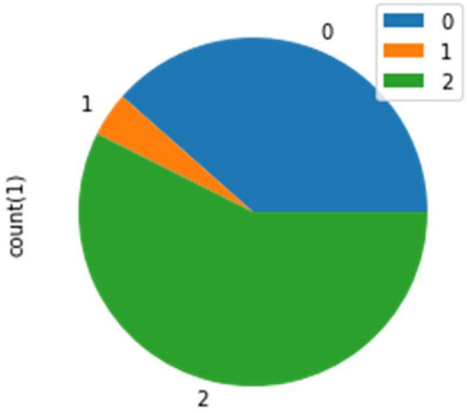
```
pandas_df = sqlDF.toPandas()
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='protocol_type', y='count(1)', kind = 'bar')
```




You can use different types of charts. See the code for Pie chart.

```
pandas_df.plot(x ='protocol_type', y='count(1)', kind = 'pie')
```

Then you can see the pie chart as below:




**Query 4:** List the name of services used for udp connections which is not host login. We use `DISTINCT` to eliminates duplicate records from the results

```
sqlDF = spark.sql("SELECT DISTINCT service from KddView where
is_host_login=0 and protocol_type='udp'")
sqlDF.show()
```

```
+--------+
| service|
+--------+
|   ntp_u|
|  tftp_u|
|domain_u|
|   other|
| private|
+--------+
```

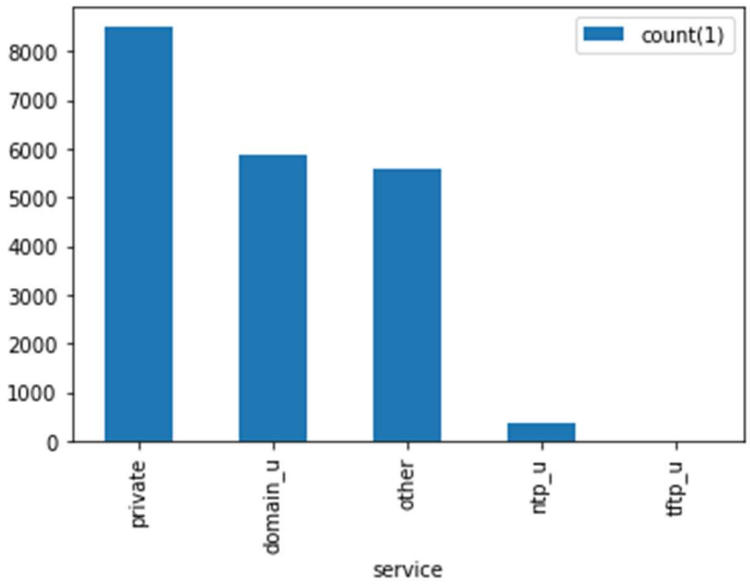
**Query 5:** List the number of different services used for udp connections which is not host login.

```
sqlDF = spark.sql("SELECT service, count(*) from KddView where
is_host_login=0 and protocol_type='udp' group by service;")
sqlDF.show()
```

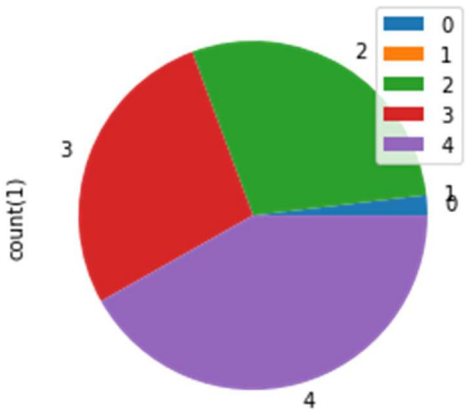
```
+--------+--------+
| service|count(1)|
+--------+--------+
|   ntp_u|     380|
|  tftp_u|       1|
|domain_u|    5863|
|   other|    5598|
| private|    8512|
+--------+--------+
```

Visual representations:

```
pandas_df = sqlDF.toPandas()
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='service', y='count(1)', kind = 'bar')
```




```
pandas_df.plot(x ='service', y='count(1)', kind = 'pie')
```




**Query 6:** List the different types of bad connections.

```
sqlDF = spark.sql("SELECT DISTINCT connection_status from KddView
where connection_status<>'normal.'")
sqlDF.show()
```

```
+-----------------+
|connection_status|
+-----------------+
|     warezmaster.|
|           smurf.|
|             pod.|
|            imap.|
|            nmap.|
|    guess_passwd.|
|         ipsweep.|
|       portsweep.|
|           satan.|
|            land.|
|      loadmodule.|
|       ftp_write.|
| buffer_overflow.|
|         rootkit.|
|     warezclient.|
|        teardrop.|
|            perl.|
|             phf.|
|        multihop.|
|         neptune.|
+-----------------+
only showing top 20 rows
```

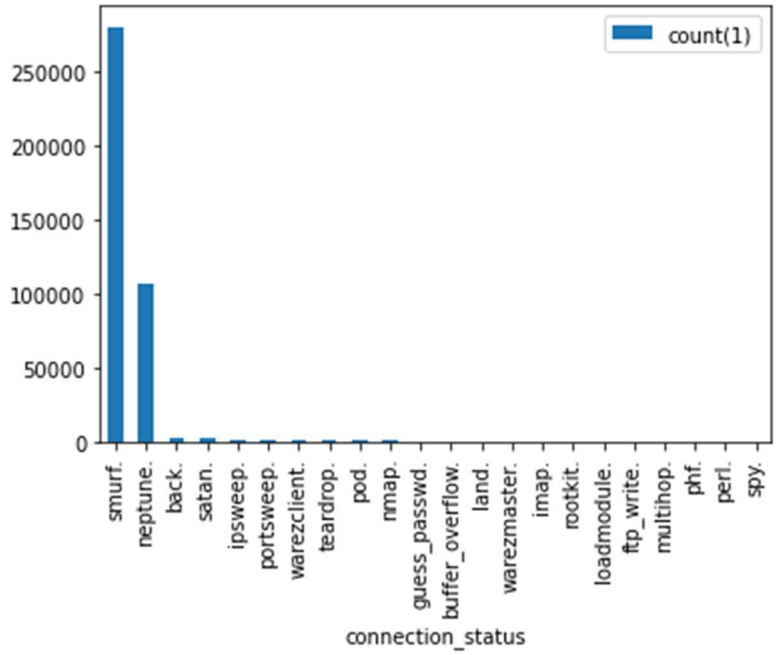
**Query 7:** List the number of different types of bad connections.

```
sqlDF = spark.sql("SELECT connection_status, count(*) from KddView
where connection_status<>'normal.' GROUP BY connection_status")
sqlDF.show()
```

```
+-----------------+--------+
|connection_status|count(1)|
+-----------------+--------+
|     warezmaster.|      20|
|           smurf.|  280790|
|             pod.|     264|
|            imap.|      12|
|            nmap.|     231|
|    guess_passwd.|      53|
|         ipsweep.|    1247|
|       portsweep.|    1040|
|           satan.|    1589|
|            land.|      21|
|      loadmodule.|       9|
|       ftp_write.|       8|
| buffer_overflow.|      30|
|         rootkit.|      10|
|     warezclient.|    1020|
|        teardrop.|     979|
|            perl.|       3|
|             phf.|       4|
|        multihop.|       7|
|         neptune.|  107201|
+-----------------+--------+
only showing top 20 rows
```

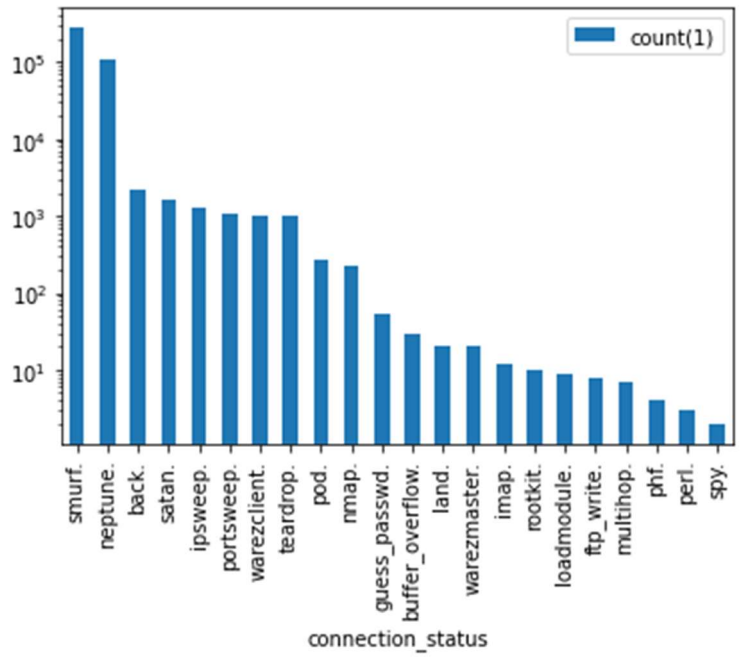
Visual representations:

```
pandas_df = sqlDF.toPandas()
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='connection_status', y='count(1)', kind = 'bar')
```




Let's use logarithmic scale for Y axis.

```
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='connection_status', y='count(1)', kind = 'bar', logy=True)
```




**Query 8:** List the services based on the received destination packet's bytes and the `tcp` protocol.

```
sqlDF = spark.sql("SELECT DISTINCT service from KddView where
dst_bytes>1000 and protocol_type='tcp'")
```

```
sqlDF.show()
```

Since there is only one value/column, we cannot use 2D plots.

```
+--------+
| service|
+--------+
|  telnet|
|     ftp|
|     X11|
|   pop_3|
|   login|
|    smtp|
|    http|
|     IRC|
|ftp_data|
|   imap4|
|     ssh|
+--------+
```

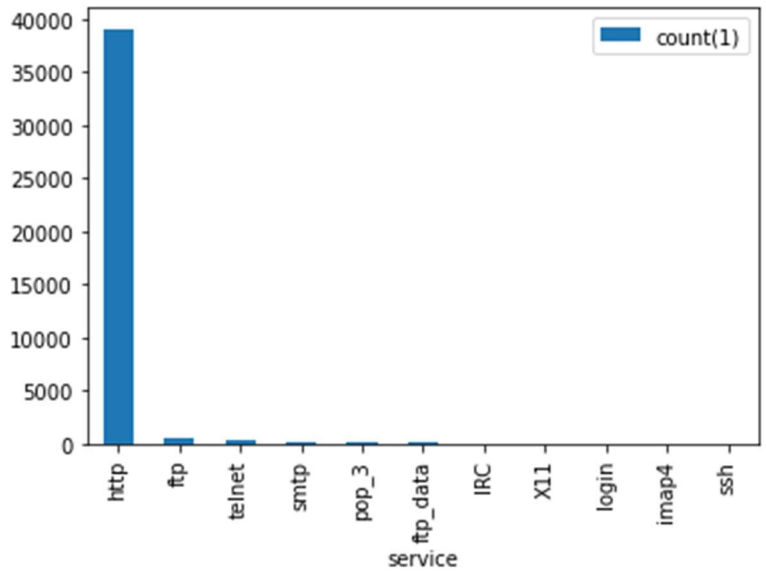
**Query 9:** List the number of services based on the received destination packet's bytes and the `tcp` protocol.

```
sqlDF = spark.sql("SELECT service, count(*) from KddView where dst_bytes>1000 and protocol_type='tcp' GROUP BY service")
```

```
sqlDF.show()
```

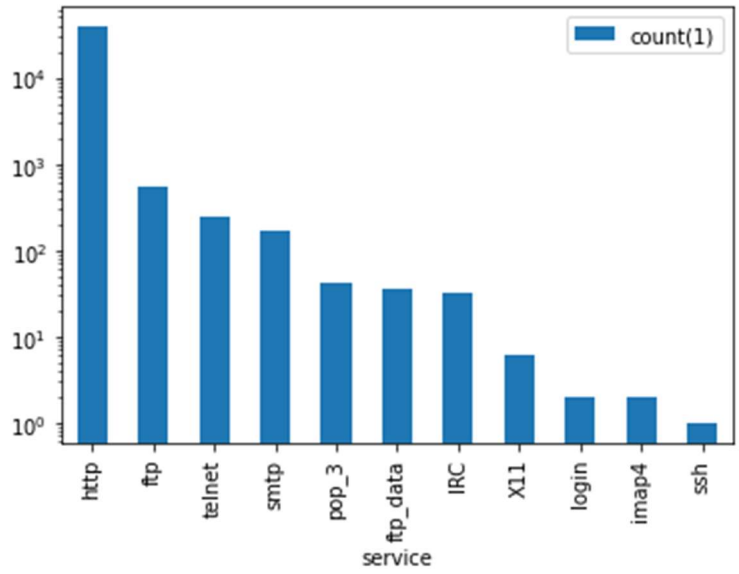
```
+--------+--------+
| service|count(1)|
+--------+--------+
|  telnet|     239|
|     ftp|     551|
|     X11|       6|
|   pop_3|      42|
|   login|       2|
|    smtp|     168|
|    http|   39047|
|     IRC|      32|
|ftp_data|      36|
|   imap4|       2|
|     ssh|       1|
+--------+--------+
```

```
pandas_df = sqlDF.toPandas()
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='service', y='count(1)', kind = 'bar')
```




Let's use logarithmic scale for Y axis.

```
pandas_df.sort_values(by='count(1)',ascending=False).plot(x
='service', y='count(1)', kind = 'bar', logy=True)
```

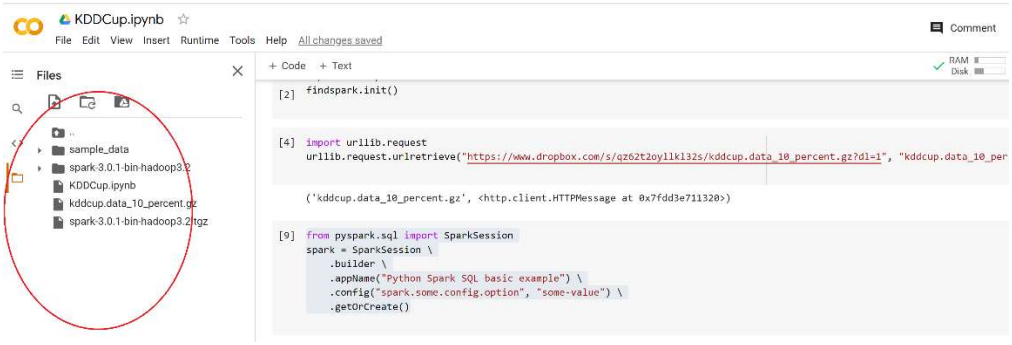



## Task 5: How to submit your work

You need to extract the html file of your work and then submit it though the submission link provided in Moodle. Here is how you can save your file as html.

### Google Colab

- Download your file through `File - > Download .ipynb`
- Drag and drop the downloaded file to the file repository of Google Colab

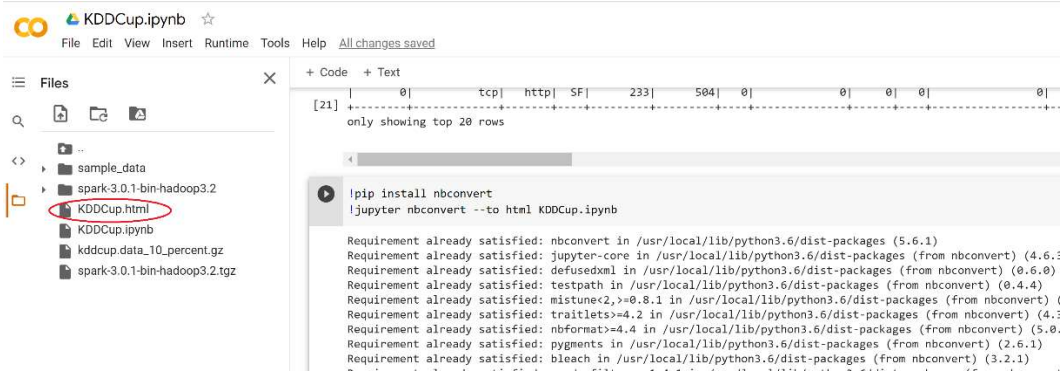



- Add the following commands to install `nbconvert` package to convert your `.ipynp` file to `html` file

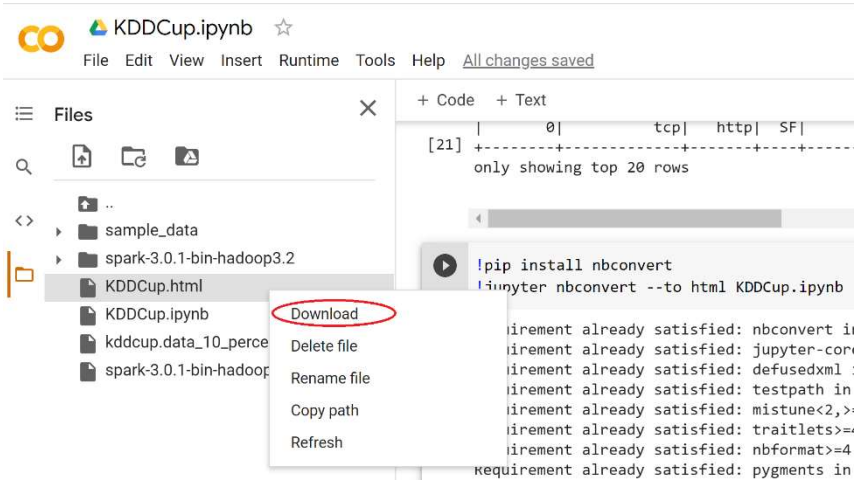
```
!pip install nbconvert
!jupyter nbconvert --to html KDDCup.ipynb
```

`KDDCup.ipynb` is the name of your file.

- You can find `KDDCup.html` the file repository.




- Right-click on `KDDCup.html` and download it.




- Submit the `KDDCup.html` through the submission link.

### Jupyter in VMWare

- Go to `File -> Download as -> html`
- You can find `html` file in the download directory

## Additional Task: Make your own Queries

It is your turn to practice more queries on KDDCup dataset using Analytic and Mathematical and many more functions. For instance, provide useful information about the number of attacks (and sub-attacks) and normal connections. You can summarize the data based on the several features, such as the status of the connections, protocol types, services, server error rates, etc. You need to create appropriate visualizations to present your findings numerically and graphically.